(https://www.facebook.com/wolfgang.ewald.7503)
(https://www.instagram.com/wolfgang.ewald.7503/?hl=de)



(https://wolles-elektronikkiste.de/en)

Wolles Elektronikkiste (https://wolles-elektronikkiste.de/en)

ABOUT ME AND THE BLOG (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/ABOUT-ME-AND-THE-BLOG)

HOME (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/)

Die wunderbare Welt der Elektronik

ABOUT THE ENGLISH VERSION (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/A-FEW-COMMENTS-ON-THE-ENGLISH-VERSION)

- Home (https://wolles-elektronikkiste.de/en)
- A few comments on the English version (https://wolles-elektronikkiste.de/en/a-few-comments-on-the-english-version)
- About me and the blog (https://wolles-elektronikkiste.de/en/about-me-and-the-blog)
- Comment Subscriptions (https://wolles-elektronikkiste.de/en/comment-subscriptions)
- Cookie Policy (https://wolles-elektronikkiste.de/en/cookie-policy)
- Disclaimer (https://wolles-elektronikkiste.de/en/disclaimer)
- Legal notice (https://wolles-elektronikkiste.de/en/legal-notice)
- List of posts (https://wolles-elektronikkiste.de/en/list-of-blog-posts)
- News Ticker (https://wolles-elektronikkiste.de/en/news-ticker)
- Support me (https://wolles-elektronikkiste.de/en/support-me)

LIST OF POSTS (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/LIST-OF-BLOG-POSTS)

NEWS (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN?PAGE_ID=14004)

DISCLAIMER (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/DISCLAIMER)

ENGLISH(HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/USING-LORA-WITH-THE-EBYTE-E220-E22-AND-E32-SERIES)

Using LoRa with the EByte E220, E22 and E32 series

POSTED ON MARCH 23, 2024 (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/USING-LORA-WITH-THE-EBYTE-E220-E22-AND-E32-SERIES) BY WOLFGANG EWALD (HTTPS://WOLLES-ELEKTRONIKKISTE.DE/EN/AUTHOR/W0ELLELLE_EWOOD)



About this Post

This article is once again about radio, more precisely about LoRa technology using the example of the Ebyte (https://www.cdebyte.com/) E220, E22 and E32 module series. The example sketches are written for the E220 modules, but can be transferred relatively easily, especially as I list the essential points that need to be adapted. Please forgive me for not having written three versions of each sketch.

Here is an overview of the post:

- Legal – please note
- What is LoRa, LoRaWAN and LPWAN?
- LoRA Modules
- Ebyte LoRa Modules
  - Overview
  - Naming
  - Technical Data (Example E220-900T22D)
  - Pinout LoRa E220, E22, E32 Series
- Libraries for the Ebyte E220, E22 and E32 Series
- Circuit for the Arduino Nano
- How to make Settings
  - Configuration Sketch using the E220 Module Series
  - Configuration Sketches for the E22 and E32 series
  - Settings in Detail
- Response Container and Response Status
- Transparent Transmission Mode
- Sending and receiving Structures
- Fixed Transmission Mode
- RSSI Received Signal Strength Indicator
- Using Wake On Radio (WOR)
- Waking up the MCU with WOR
- Range Test
- Appendices – Transceiver Sketches with Settings for E220, E22, E32

Legal – please note!

Important to know:

- Radio transmission is regulated by law in almost every country in the world, e.g. regarding the permitted frequencies, the transmission power or the duty cycle (percentage of the actual transmission time).
- Not everything you can buy in your country can be used without restrictions – and some things can’t be used at all!

In Germany, the rules for radio are set by the Federal Network Agency. You can find the relevant details here (https://www.bundesnetzagentur.de/DE/Fachthemen/Telekommunikation/Frequenzen/Allgemeinzuteilungen/start.html) in the section “Funkanlagen mit geringer Reichweite”. Hobbyists mainly use the frequencies around 433 MHZ, 868 MHz, 915 MHz and 2.4 GHz – but they are not free of restrictions. **You are responsible for complying with the regulations in your country**.

What is LoRa, LoRaWAN and LPWAN?

If you are dealing with LoRa, you will also come across the terms LoRaWAN and LPWAN. LoRa stands for **Long Range**, i.e. it is a radio technology for comparatively long distances. LoRa (like Sigfox (https://wolles-elektronikkiste.de/en/arduino-iot-cloud-part-2-arduino-sim-and-sigfox)) belongs to the group of LPWAN radio technologies, which in turn is the abbreviation for Low-Power Wide Area Network. This name thus contains a second characteristic of this technology, namely its low-energy consumption. While LoRa refers to the special radio technology, LoRaWAN is a communication protocol based on it.

I am neither a physicist nor a radio engineer. That’s why I prefer to hold back on explaining exactly how LoRa works. Just this much: It is a special modulation technology that achieves the long-range and low-energy consumption. The data transfer rates are comparatively low.

LoRa Modules

Many different LoRa modules are offered in popular online stores such as Amazon or AliExpress. These also include ready-made solutions, e.g. ESP32-based boards with OLED displays. Originally, I had planned to give a broader overview of common modules and libraries. However, I then realized that the topic is simply too extensive to cover everything in one post. I will therefore initially limit myself to the E220, E22 and E32 series from Ebyte. They are close relatives that can be programmed in almost the same way. I may look at other modules in a later post.

Ebyte LoRa Modules

Overview

Ebyte, more precisely Chengdu Ebyte Electronic Technology Co. Ltd (https://www.cdebyte.com/), offers various LoRa modules. If you are looking for specific models, take a look here (https://www.cdebyte.com/Module-Lora). I have summarized the most important differences between the module series here:

	E32-series	E22-series	E220-series
Chip	SX1276 / SX1278	SX1262 / SX1268	LLCC68
Range [km]	5 - 8	5 - 12	7 - 10
Repeater	No	Yes	No
Remote Configuration	No	Yes	No
Max Message Size [byte]	58	240	200
Max Air Data Rate [bps]	19.2k	62.5k	62.5k
Buffer Size [byte]	512	1000	400
Encryption	One Standard	Configurable	Configurable
RSSI (Received Signal Strength Indicator)	No	Yes	Yes
FEC Control (Forward Error Correction)	Yes	No	No
LBT (Listen Before Talk)	No	Yes	Yes

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/e220_e32_e22_comparison-2-1024x514.png)

Overview of the E220, E22 and E32 series

The modules all have the same pinout and are very similar in operation.

An overview of the SX1262, SX1276 and SX1278 chips can be found here (https://www.cdebyte.com/news/580). All chips and modules have their advantages and disadvantages.

Naming

The modules are named according to the scheme E*aaa*-*bbb*T*ccd*, e.g. E220-900T22D. The parameters are:

- aaa: the “family name”,
- bbb: represents the radio frequency,
- cc: represents the signal strength in dBm,

- d: stands for the design (DIP or SMD).

Here, as an example, is an overview of some representatives of the E220 series:

Model No.	Chip Scheme	Frequency Hz	Tx power dBm	Distance km	Package	Size mm	Interface
E220-400T22S	LLCC68	433/470M	22	5	SMD	16*26	TTL
E220-400T30S	LLCC68	433/470M	30	10	SMD	20*40.5	TTL
E220-900T22S	LLCC68	868/915M	22	5	SMD	16*26	TTL
E220-900T30S	LLCC68	868/915M	30	10	SMD	20*40.5	TTL
E220-400T22D	LLCC68	433/470M	22	5	DIP	21*36	TTL
E220-400T30D	LLCC68	433/470M	30	10	DIP	24*43	TTL
E220-900T22D	LLCC68	868/915M	22	5	DIP	21*36	TTL
E220-900T30D	LLCC68	868/915M	30	10	DIP	24*43	TTL

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/E220_series-1024x344.png)

Model overview of the LoRa E220 series (source: E220-900T22D data sheet)

Technical Data (Example E220-900T22D)

You can find data sheets and manuals on the Chengdu Ebyte website (https://www.cdebyte.com/Module-Lora). As an example, I have the technical data of the E220-900T22D model for you here:

Table 2-2 Working parameter table

Main Parameter		Performance			Remark
		Min.	Typ.	Max.	
Operating voltage (V)		2.3	5	5.5	≥3.3V ensures output power
Communication level (V)		-	3.3	-	For 5V TTL, it may be burning down
Operating temperature (℃)		-40	-	+85	Industrial design
Operating frequency (MHz)		850.125	-	930.125	Support ISM band
Power consumption	TX current(mA)	-	110	-	Instant power consumption @22dBm
	RX current (mA)	-	16.8	-	5V Voltage
	Sleep current (μA)	-	5	-	Software shutdown
Max Tx power (dBm)		21. 5	22. 0	22. 5	-
Receiving sensitivity (dBm)		-146	-147	-148	Air data rate 2.4kbps
Air data rate (bps)		2.4k	2.4k	62.5k	To control via user's program
Distance for reference		5km			Test condition : clear and open area, antenna gain: 5dBi, antenna height: 2.5m, air data rate: 2.4kbps
TX length		200 Byte			Can be configured via command as 32/64/128/200 bytes per packet to transmit
Buffer		400 Byte			-
Modulation		LoRa			New generation LoRa modulation technology
Communication interface		UART Serial Port			TTL Level
Package		DIP			-
Size		21*36mm			-
Antenna		SMA-K			50 ohm impedance

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/e220_tech_data_1.png)

Technical data E220-900T22D LoRa module (source: E220-900T22D data sheet)

I found a data sheet for the same module elsewhere, but with slightly different information on the optimum voltage supply:

Table 2-2 Working parameter table

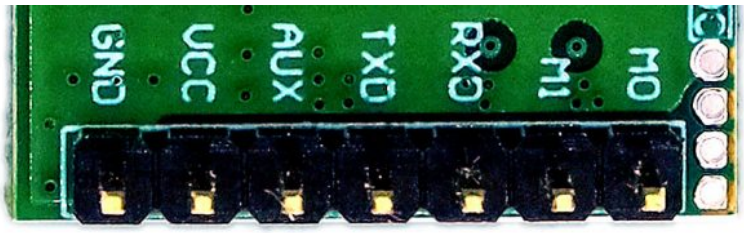
Main Parameter	Performance			Remark
	Min.	Typ.	Max.	
Operating voltage (V)	3.0	5	5.5	≥5.0 V ensures output power
Communication level (V)	-	3.3	-	For 5V TTL, it may be burning down
Operating temperature (℃)	-40		+85	Industrial design

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/e220_tech_data_2-1.png)

Version 2 of the E220-900T22D LoRa module.

If in doubt, try out whether you achieve better results with 5 volts. However, it is important to note that not all pins can handle 5 volts. For 5 volt MCUs, you should therefore use level shifters or voltage dividers.

Pinout LoRa E220, E22, E32 Series



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/pins_e220_lora-1024x189.jpg)

Pinout LoRa E220, E22 and E32

- **GND / VCC**: Power supply
- **AUX**: Indicates the status of the data buffers for sending and receiving and is used for the self-check.
- **RX / TX**: Serial communication
- **M0 / M1 (E220)**: Control of the four operating modes:
  1. **M0 = LOW, M1 = LOW**: Normal mode
  2. **M0 = HIGH, M1 = LOW**: WOR Transmitter
  3. **M0 = LOW, M1 = HIGH**: WOR Receiver
  4. **M0 = HIGH, M1 = HIGH**: Deep sleep
- **M0 /M1 (E22)**: Control of the four operating modes:
  1. **M0 = LOW, M1 = LOW**: Normal mode
  2. **M0 = HIGH, M1 = LOW**: WOR mode (transmitter/receiver)
  3. **M0 = LOW, M1 = HIGH**: Configuration mode
  4. **M0 = HIGH, M1 = HIGH**: Deep sleep
- **M0 / M1 (E32)**: Control of the four operating modes:
  1. **M0 = LOW, M1 = LOW**: Normal mode
  2. **M0 = HIGH, M1 = LOW**: Wake-up mode (transmitter)
  3. **M0 = LOW, M1 = HIGH**: Power saving
  4. **M0 = HIGH, M1 = HIGH**: Sleep

Libraries for the Ebyte E220, E22 and E32 Series

I selected the libraries of Renzo Mischianti (https://mischianti.org/) to control the modules. You can download them here from GitHub:

- EByte_LoRa_E220_Series_Library (https://github.com/xreef/EByte_LoRa_E220_Series_Library)
- EByte_LoRa_E22_Series_Library (https://github.com/xreef/EByte_LoRa_E22_Series_Library)
- LoRa_E32_Series_Library (https://github.com/xreef/LoRa_E32_Series_Library)

... or install via the library manager of the Arduino IDE.

In addition, the industrious Renzo Mischianti has created a multipart tutorial for each of the module series. Here are the links to the first parts: E220-Tutorial (https://mischianti.org/ebyte-lora-e220-llcc68-device-for-arduino-esp32-or-esp8266-specs-and-basic-use-1/), E22-Tutorial (https://mischianti.org/ebyte-lora-e22-device-for-arduino-esp32-or-esp8266-specs-and-basic-usage-1/), E32-Tutorial (https://mischianti.org/lora-e32-device-for-arduino-esp32-or-esp8266-specs-and-basic-usage-part-1/).

My motivation for writing my own article was to summarize things a little more compactly and to compare the module series. However, it is definitely worth taking a look at the tutorials. They offer a lot of additional information, such as example sketches and circuit diagrams for other boards.

The tutorials and libraries for the various series are structured in the same way. This means you can quickly find your way around if you change the module series.

Circuit for the Arduino Nano

In this article, I use the classic Arduino Nano as the controlling board. I connected it to the E220 module as follows (identical for E22 and E32):

E220 Pin	Arduino Pin	Pullup-Resistor	Levelshifter
VCC	5V		
GND	GND		
AUX	D3 (Interruptpin)	X	
TX	D4 (SoftwareSerial RX)	X	
RX	D5 (SoftwareSerial TX)	X	X
M1	D6		X
M0	D7		X

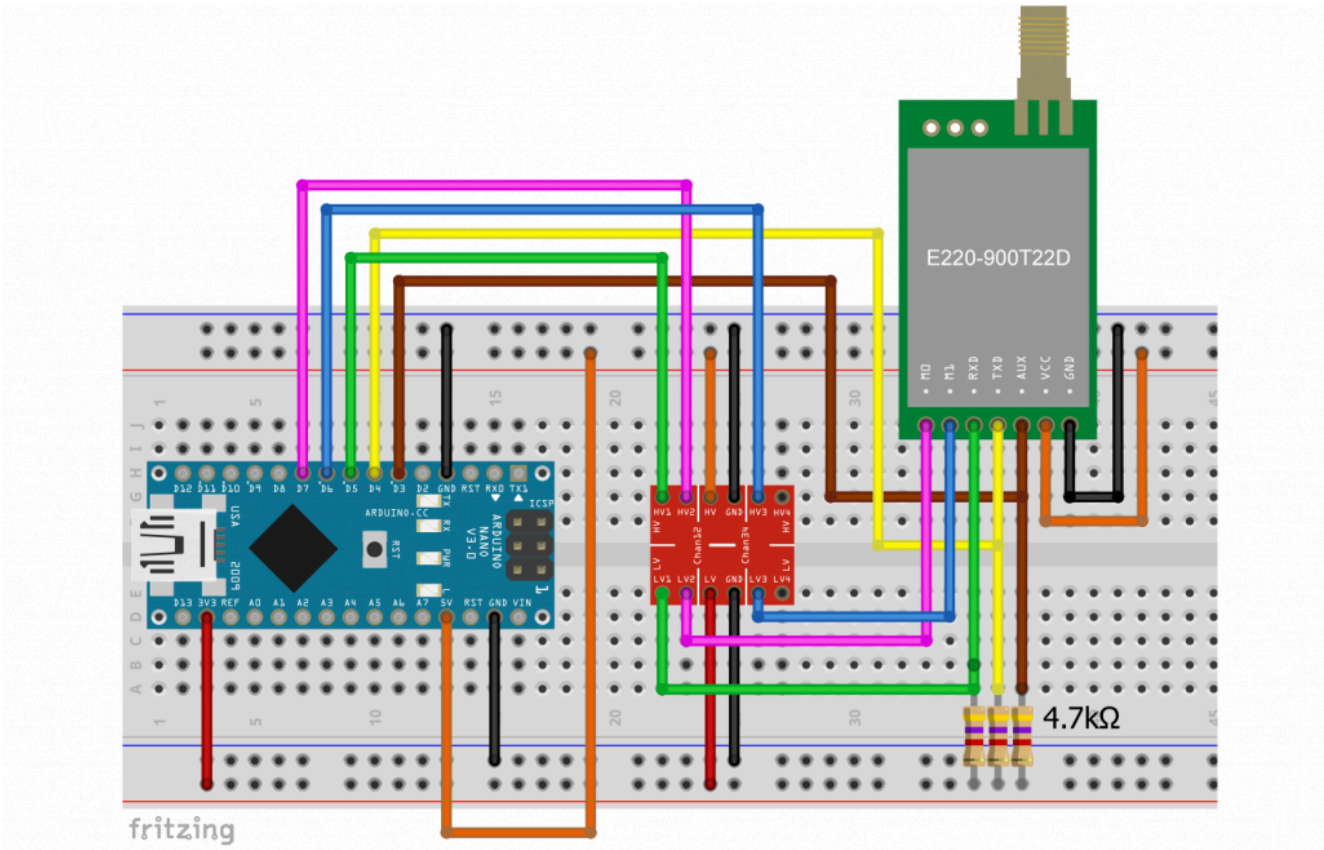
(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/e220_lora_arduino_connect.png)

Connection of the E220 module to the Arduino Nano

A few comments on this:

- If you do not want to use the WOR (Wake On Radio) or power-down modes, you can connect M0 and M1 to GND.
- In principle, you can also leave AUX unconnected. In this case, the microcontroller does not know when the data transfer is complete, but the library provides a sufficient waiting time.
- When using a 5 volt board, you must use a level shifter or a voltage divider (e.g. 2 kΩ / 1 kΩ) for connecting M0, M1, RX to the board pins.
- AUX, RX and TX require a pull-up resistor.
- Example circuits for other MCU boards can be found in the tutorials by Renzo Mischianti.
- The antennas for the E220-900T22D and E32-433T30D modules I used were special versions with a magnetic base for 868 MHz and 433 MHz. These are available for < €10 in online stores.

This is what my setup looked like on the breadboard:



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/Lora_E220_Arduino_Nano-1024x657.png)

LoRa E220 module connected to an Arduino Nano on the breadboard

How to make Settings

Configuration Sketch using the E220 Module Series

The libraries are equipped with separate example sketches for setting up and operating the modules. Use setConfiguration.ino to make the settings, and getConfiguration.ino to query them. **These settings are retained even if the power supply is disconnected, provided you select the parameter WRITE_CFG_PWR_DWN_SAVE when saving the configuration** (line 62 in the next sketch).

When you open setConfiguration.ino (https://github.com/xreef/EByte_LoRa_E220_Series_Library/blob/master/examples/01_setConfiguration/01_setConfiguration.ino), you may be overwhelmed by its size. However, most of it is commented out, and many things are repeated. The upper section contains various examples of object creation for different boards. The middle section contains ready-made configuration examples that you can easily uncomment. At the end you will find the functions for outputting the settings.

I have reduced setConfiguration.ino (E220 module) to the essentials for use on an Arduino Nano. But I have also added a few lines:

- Lines 1 to 5 are used to output the correct frequency. Just uncomment the line with the correct frequency range.
- Lines 6 and 7 are relevant for selecting the signal strength.
- Line 8 ensures that the settings are output in detail.
- Lines 57 and 58 allow you to encrypt your messages individually.

setConfiguration_modified.ino

```
1.  // #define FREQUENCY_433 // default value without set
2.  // #define FREQUENCY_170
3.  // #define FREQUENCY_470
4.  #define FREQUENCY_868
5.  // #define FREQUENCY_915
6.  // #define E220_22 // default value without set
7.  // #define E220_30 // uncomment in case you use an E220...T30D or E220..
8.  #define LoRa_E220_DEBUG  // for printing the settings
9.  #include "LoRa_E220.h"
10.
11.  SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX -->
12.  LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1
13.  // LoRa_E220 e220ttl(4, 5, 3, 7, 6); // alternative function to create t
    LoRa_E220 object
14.
15.  void printParameters(struct Configuration configuration);
16.  void printModuleInformation(struct ModuleInformation moduleInformation);
```

```
17.
18. void setup() {
19.   Serial.begin(9600);
20.   while(!Serial){};
21.   delay(500);
22.
23.   Serial.println();
24.
25.
26.   // Startup all pins and UART
27.   e220ttl.begin();
28.
```

When trying things out, it can be a bit annoying to adjust the settings via a separate sketch, even if this saves resources. Perhaps you would prefer to make the settings directly in the sketches that you also use for control? And perhaps you would like to have an overview of all the setting options? Then take a look at Appendix 1 for the E220 series.

Configuration Sketches for the E22 and E32 Series

In Appendix 2 and Appendix 3 you will find the counterparts to Appendix 1 for the E22 and E32 series respectively. The sketches also provide a good overview of the differences between the three module series.

Settings in Detail

Address and Channel (configuration)

Each LoRa module has an address, which is made up of the higher byte `ADDH` and the lower byte `ADDL`. This allows you to set 65536 addresses. The address is assigned via `configuration.ADDL = ...` or `configuration.ADDH = ...`.

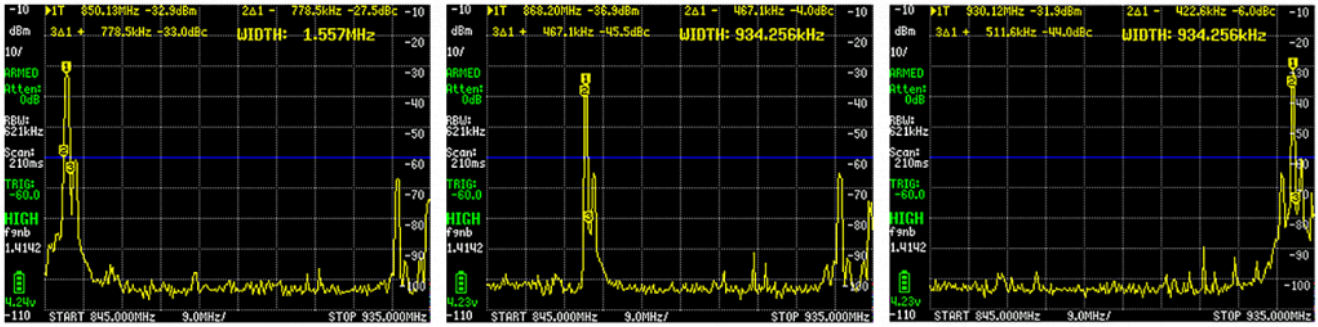
The modules I tested cover the frequencies 850 – 930 MHz and 410 to 493 MHz. Fine adjustment is made via the channel. The formula for the frequency ν is

$$\nu_{Exxx-900}\ [\mathrm{MHz}] = 850.125\ + CHANNEL$$
$$\nu_{Exxx-400}\ [\mathrm{MHz}] = 410.125 + CHANNEL$$

For the Exxx-900 modules, use `configuration.CHAN = 18` to set the frequency to 868.125 MHz. With the Exxx-400 models, you set 433.125 MHz with `configuration.CHAN = 23`.

There is a certain risk that you will transmit on a frequency that is not permitted in your country if you make the wrong selection. I'm just saying.

A check with a tinySA Spectrum Analyzer (https://www.tinysa.org/wiki/) showed a good match with the target frequency within the accuracy of this device. Here is the result for channels 0, 18 and 80 of an E220-900T22D module:



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/frequency_check_e220_900_ebyte-1024x260.png)

Checking the transmission frequency for channel 0, 18 and 80 -> 850.13 / 868.20 / 930.12 MHz

The principle of channel setting is identical for all modules.

Transmission Rate and Options (configuration.SPED)

You can set the following parameters for UART communication and data transmission over the air (example E220):

- **uartBaudRate**: Baud rate UART_BPS_xxx
  - xxx = 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200
- **airDataRate**: Data rate for radio transmission AIR_DATA_RATE_xxx
  - AIR_DATA_RATE_010_24: 2.4 kbit/s (default)
  - AIR_DATA_RATE_011_48: 4.8 kbit/s
  - AIR_DATA_RATE_100_96: 9.6 kbit/s
  - AIR_DATA_RATE_101_192: 19.2 kbit/s
  - AIR_DATA_RATE_110_384: 38.4 kbit/s
  - AIR_DATA_RATE_111_625: 62.5 kbit/s
- **uartParity**: Parity MODE_xxx;
  - MODE_00_8N1: none (default)
  - MODE_01_8O1: odd
  - MODE_10_8E1: even

The options for the airDataRate differ for the E22 and 32 series. See Appendix 2 and 3.

Regarding the data rate via radio: the lower the data rate, the higher the maximum range.

Further Transmission Options (configuration.OPTION)

Further options (example E220):

- **subPacketSetting**: Maximum data packet size, i.e. the maximum length of your message that you can send continuously in one piece.
  - SPS_200_00: 200 bytes
  - SPS_128_01: 128 bytes
  - SPS_064_10: 64 bytes
  - PLC_032_11: 32 bytes
- **RSSIAmbientNoise**: RSSI (Received Signal Strength Indicator) Ambient Noise enable
  - RSSI_AMBIENT_NOISE_DISABLED: Function is switched off
  - RSSI_AMBIENT_NOISE_ENABLED: Function is switched on
- **transmissionPower**: Transmission power POWER_xx with xx = power in dbm. The setting options depend on the model you are using.
  - xx for E220....22D: 22, 17, 13, 10
  - xx for E220....30D: 30, 27, 24, 21

The subPacketSetting and transmissionPower settings differ for the E22 and E32 series (see Appendix 2 and 3). RSSIAmbientNoise is not available on the E32 module.

Mode Settings (configuration.TRANSMISSION_MODE)

I will come back to some of these settings later. Here are the options for the E220 series:

- **enableRSSI**: Signal strength information (Received Signal Strength Indication)
  - RSSI_DISABLED: disabled
  - RSSI_ENABLED: enabled
- **fixedTransmission**: Transmission mode
  - FT_FIXED_TRANSMISSION: Transmission to a specific address / channel.
  - FT_TRANSPARENT_TRANSMISSION: Transmission to all modules with identical address and channel.
- **enableLBT**: LBT (Listen Before Talk) is a function that causes the module to wait up to two seconds for a favorable moment (with low interference) before transmitting.
  - LBT_DISABLED: Function disabled
  - LBT_ENABLED: Function enabled
- **WORPeriod**: WOR (Wake On Radio) period WOR_xxx_yyy with xxx = wake-up period in milliseconds

- xxx_yyy = 500_000, 1000_001, 1500_010, 2000_011, 2500_100, 3000_101, 3500_110, 4000_111

The WORPeriod parameter needs a little more explanation. This is the period after which the receiver wakes up to check whether a message is coming. Of course, the sender does not know when the receiver is awake and therefore sends a “preamble” with the length of the wake-up period. The WOR receiver then remains awake until the actual message is received. For this reason, the WOR Receiver and the WOR Transmitter must have the same WORPeriod set.

The WORPeriod is called wirelessWakeUpTime when using the E32 series. The selectable periods are also different. The enableRSSI and enableLBT settings are not available on the E32 modules.

Encryption (configuration.CRYPT)

All modules encrypt the messages. However, only the E22 and E220 series modules allow individual encryption. To do this, you define values for the two bytes CRYPT_H and CRYPT_L. The setting must, of course, be identical for the transmitter and receiver. A fixed encryption setting is used on the E32.

Response Container and Response Status

Before we finally come to the pratical part, I need to explain two structures defined in the library, namely ResponseContainer and ResponseStatus. They contain information about the incoming and outgoing messages.

The ResponseContainer contains the actual message `data`, the signal strength value `rssi`, and the ResponseStatus `status`.

struct_response_container

```
struct ResponseContainer {
  String data;
  byte rssi; // only E22 and E220 series!
  ResponseStatus status;
};
```

The ResponseStatus is also a structure. It contains the error code `code` and the function `getResponseDescription()`, which returns a string that translates the code comprehensibly. For example, error code 1 (E220_SUCCESS) means “Success”.

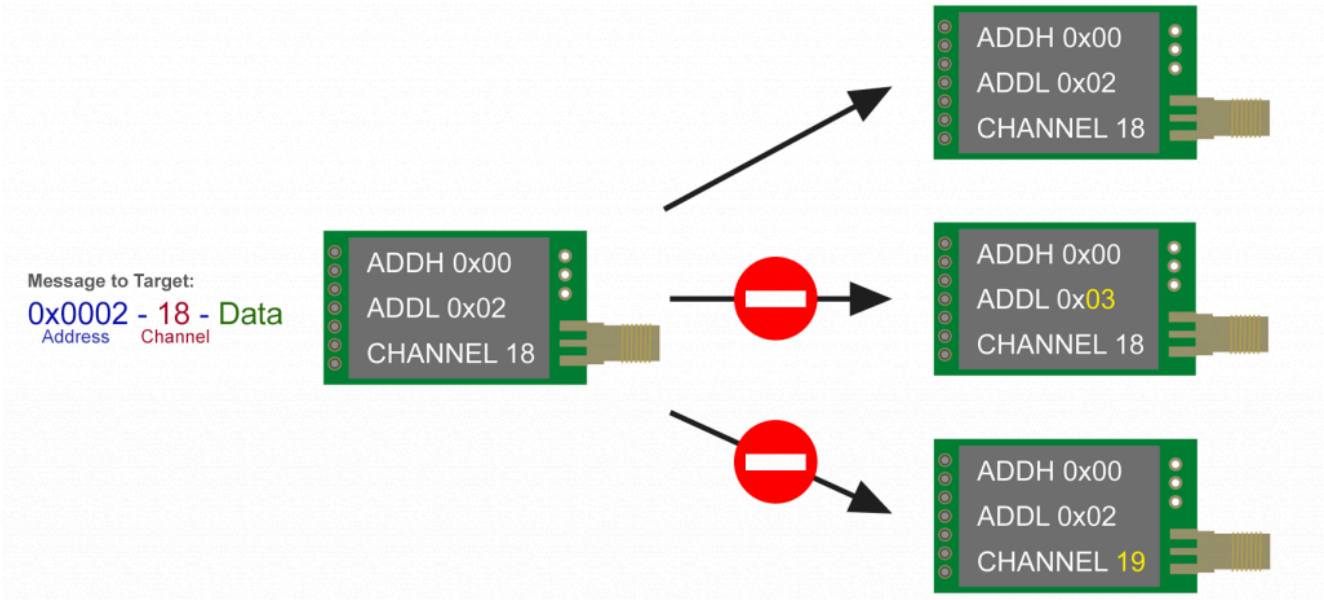
struct_response_status

```
struct ResponseStatus {
  Status code;
```

```
    String getResponseDescription() {
      return getResponseDescriptionByParams(this->code);
    }
  };
```

Transparent Transmission Mode

But now to the first sketch. We allow several (at least two) modules to communicate with each other in “Transparent Transmission Mode”. In this mode, you can reach all LoRa modules that have the same address and channel set:



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/lora_transparent_transmission_scheme-1024x468.png)

Scheme for transparent transmission mode

To try out the transparent mode, you should not normally need to make any changes to the settings. However, if you have problems, set the modules to the correct state with setConfiguration_modified.ino.

Then upload the lora_transparent.ino sketch to the microcontroller boards. You may need to adjust lines 3 and 4 for your boards. Take a look at the original example sketches in the library – there are many ready-made settings for different boards.

In most examples, we need a separate serial monitor for each board. With the old Arduino IDE 1.x, this was easy by simply calling up the IDE several times, i.e. creating several instances and selecting a separate serial port for each instance. The IDE 2.x does not seem to allow multiple instances. You can help yourself by saving the sketch under different names, opening the versions, and then setting a separate port for each sketch.

lora_transparent.ino

```
1.  #include "LoRa_E220.h"
2.
3.  SoftwareSerial mySerial(4,5);
4.  LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1
5.
6.  void setup() {
7.    Serial.begin(9600);
8.    delay(500);
9.
10.   Serial.println("Hi, I'm going to send message!");
11.
12.   // Startup all pins and UARTD
13.   e220ttl.begin();
14.
15.   // Send message
16.   ResponseStatus rs = e220ttl.sendMessage("Hello, world?");
17.   // Check if there is some problem of successfully send
18.   Serial.println(rs.getResponseDescription());
19. }
20.
21. void loop() {
22.     // If something available
23.   if (e220ttl.available()>1) {
24.       // read the String message
25.     ResponseContainer rc = e220ttl.receiveMessage();
26.     // Is something goes wrong print error
27.     if (rc.status.code!=1){
28.         Serial.println(rc.status.getResponseDescription());
29.     }else{
```

Apart from the library to be integrated and the object creation (line 4), the sketch works in the same way with the E22 and E32 boards.

Explanations for lora_transparent.ino

First create the object e220ttl and initialize your module with `e220ttl.begin()`.

You send messages using `sendMessage()`. As you can see, you do not enter an address or channel in transparent mode. The module automatically sends to the modules with the same settings. The return value of `sendMessage()` is a structure of the type `ResponseStatus`, which I explained above. You can get a readable translation of the status code with `getResponseDescription()`. Ideally, this is “Success”. However, “Success” only means that the message was sent successfully and not that it was received.

You can check whether a message has been received on the receiver side with `e220ttl.available()`. You are reading the message using `receiveMessage()`. More precisely, `receiveMessage()` returns a structure of the type `ResponseContainer`, which contains the actual message in the element `data`.

The `ResponseContainer` "rc" in turn contains the `ResponseStatus` "rc.status". You check with

```
if (rc.status.code != 1) {....
```

whether an error has occurred. Perhaps the variant

```
if (rc.status.getResponseDescription() = "Success") {...
```

would be a little easier to understand, but I wanted to stay close to the original library sketches.

If everything is OK, the message is displayed, if not, the corresponding error message appears.

Use `if (Serial.available(){...` to check whether an input has been made via the serial monitor. If this is the case, the entry is read and sent.

Output lora_transparent.ino

I used three modules. Below, you can see the output of the first activated module. The two "Hello, world?" messages were sent via the setup of the other modules. I sent the two other messages "manually" from the other modules by entering them in the serial monitor.

```
Hi, I'm going to send message!
Success
Hello, world?
Hello, world?
Hi - message from no. 1
Hi  - message from no. 2
```

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/output_lora_transparent.png)

Output lora_transparent. ino

Sending and receiving Structures

In most cases, you probably do not want to transmit character strings, but data, such as that of a weather station. In the following example, we send the humidity (integer), the temperature (float) and the rain status (bool) every five seconds. To send this data as a "package", we define the structure `weatherData`.

For this example, we will remain in transparent mode. The two sketches for the transmitter and receiver only need to be changed for the E22 and E32 modules regarding the library to be integrated and the object creation.

Transmitter

The sketch for the transmitter should be largely self-explanatory:

lora_struct_transmitter.ino

```
#include "LoRa_E220.h"

SoftwareSerial mySerial(4,5);
LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1

struct weatherData {
  int humidity;
  float temperature;
  bool rain;
};

weatherData currentWeather = {50, 20.0, false};

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Hi, I'm going to send message!");

  // Startup all pins and UARTD
  e220ttl.begin();

  // Send message
  ResponseStatus rs = e220ttl.sendMessage("Hello, world?");
  // Check if there is some problem of successfully send
  Serial.println(rs.getResponseDescription());
}

void loop() {
```

Regarding the sending process, the only difference to the previous sketch is that we do not pass `sendMessage()` a string, but a structure (as reference using "&") and the size of the structure.

Receiver

To be able to process the structure to be received, we use the structure `ResponseStructContainer` instead of `ResponseContainer`, which is defined as follows:

ResponseStructContainer

```
struct ResponseStructContainer {
  void *data;
  byte rssi;
  ResponseStatus status;
  void close() {
    free(this->data);
  }
};
```

The data type `void*` is an untyped pointer that many readers may not be familiar with. In contrast to the usual pointers such as `int*`, its meaning, i.e. the data type, must first be assigned to it.

Here is the sketch:

lora_struct_receiver.ino

```
#include "LoRa_E220.h"

SoftwareSerial mySerial(4,5);
LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1

struct weatherData {
  int humidity;
  float temperature;
  bool rain;
};

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Hi, waiting for weather data...");
  // Startup all pins and UARTD
  e220ttl.begin();
}

void loop() {
  if (e220ttl.available()>1) {
      // read the String message
    ResponseStructContainer rsc = e220ttl.receiveMessage(sizeof(weatherData));
    weatherData currentWeather = *(weatherData*) rsc.data;
    Serial.print("Humidity     [%]: ");
    Serial.println(currentWeather.humidity);
    Serial.print("Temperature [°C]: ");
```

The line:

```
ResponseStructContainer rsc = e220ttl.receiveMessage(sizeof(weatherData));
```

reads the received message and saves it in `ResponseStructContainer` "rsc". Please note that we must pass the size of the expected data packet to `receiveMessage()`.

The actual data is stored as data type `void*` in `rsc.data`. To be able to do something with this, we cast the data type `void*` into a pointer with `(weatherData*)`, which points to the structure of type `weatherData`. To save the data in the structure `currentWeather`, we have to dereference the pointer with another `*`. So:

```
weatherData currentWeather = *(weatherData*) rsc.data;
```

The output is still missing. It's a bit boring, of course, because we always send the same data. But this is only about the principle.

```
Hi, waiting for weather data...
Humidity     [%]: 30
Temperature [°C]: 23.70
Rain            : no
```

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/output_lora_transparent_struct_receiver.png)

Output lora_struct_receiver.ino

Fixed Transmission Mode

You set the fixed transmission mode by changing the line

```
configuration.TRANSMISSION_MODE.fixedTransmission = FT_TRANSPARENT_TRANSMISSION;
```

to:

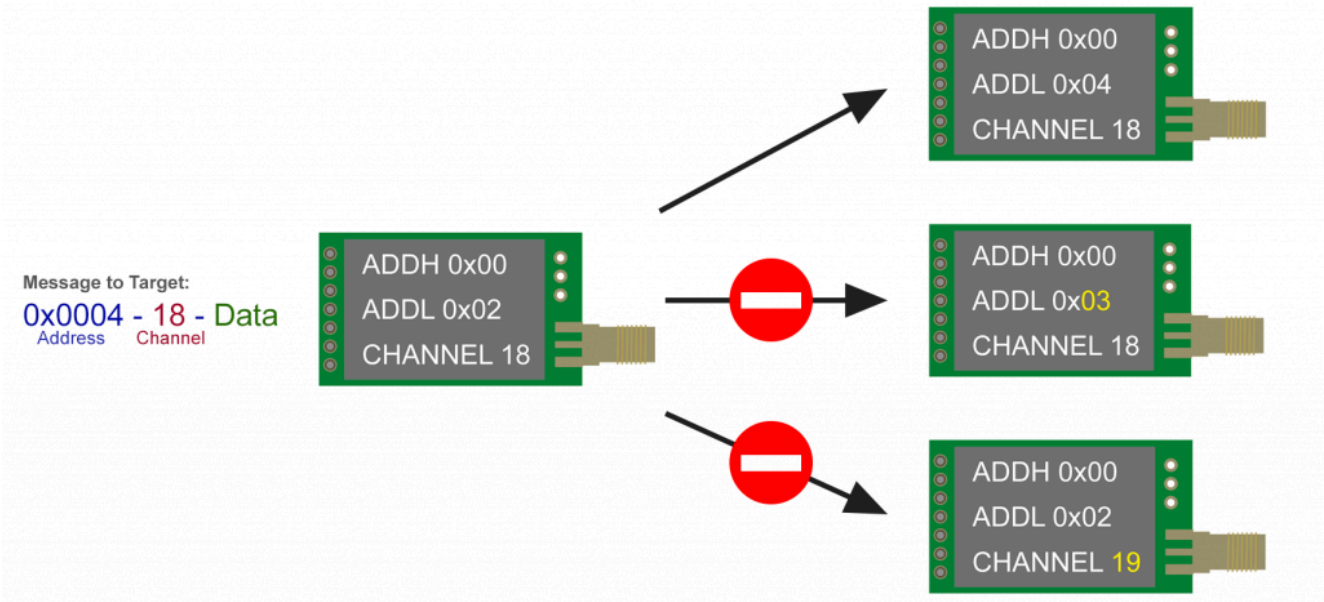
```
configuration.TRANSMISSION_MODE.fixedTransmission = FT_FIXED_TRANSMISSION;
```

There are two options for sending messages in fixed transmission mode:

1. Exclusive transmission to the modules with a specific address on a specific channel:
   - Function: `sendFixedMessage(ADDH, ADDL, channel, message);`
   - The transmitting module itself may have a different address and a different channel set.
2. Sending to all modules on a specific channel, regardless of their address ("broadcasting"):

- Function: `sendBroadcastFixedMessage(channel, message)`
  - which corresponds to: `sendFixedMessage(0xFF, 0xFF, channel, message)`, as 0xFFFF is the broadcasting address.
- The transmitting module may also have a different channel set here.

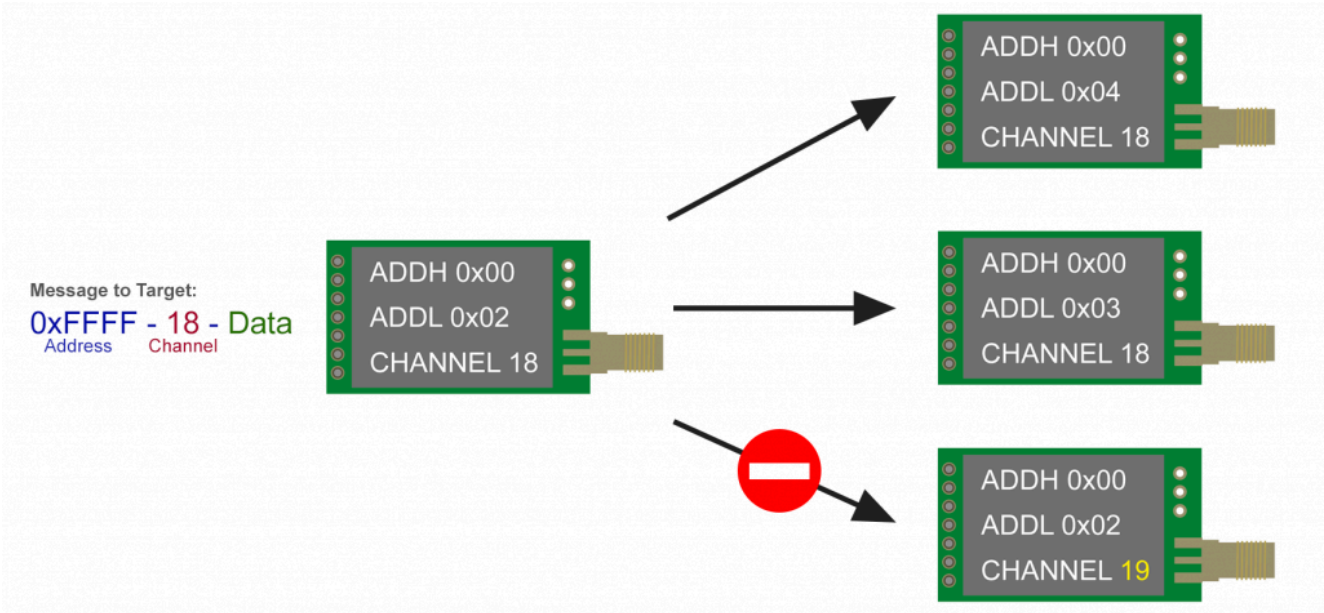
I have two diagrams to illustrate this. Here first for sending to **a specific address**:



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/lora_fixed_transmission_scheme-1024x470.png)

Scheme for fixed transmission mode

And here is the diagram for **broadcasting** in fixed transmission mode:



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/lora_fixed_transmission_broadcast_scheme-1024x476.png)

Scheme for broadcasting in fixed transmission mode

Example Sketch Fixed Transmission Mode

To test the fixed transmission mode, I used three modules with the following settings:

1. ADDH = 0x00, ADDL = **0x01**, Channel = 18
2. ADDH = 0x00, ADDL = **0x02**, Channel = 18
3. ADDH = 0x00, ADDL = **0x03**, Channel = 18

I have uploaded the following sketch to the three controlling Arduino boards, whereby I have only adapted the “greeting” in line 16 and the switch construction from line 38.

The sketch allows messages entered via the serial monitor to be sent to specific addresses (input: “x,message” with x = 1,2,3) or to all modules via broadcasting (input: “18,message”).

The sketch only needs to be adapted for use on E22 and E32 modules regarding the library and object creation.

lora_fixed_transmission_no_1.ino

```
1.  #include "LoRa_E220.h"
2.
3.  SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX --
    > e220 RX
4.  LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1
5.
6.  void setup() {
7.    Serial.begin(9600);
8.    delay(500);
9.
10.   Serial.println("Hi, I'm going to send message!");
11.
12.   // Startup all pins and UART
13.   e220ttl.begin();
14.
15.   // Send message
16.   ResponseStatus rs = e220ttl.sendBroadcastFixedMessage(18,"Hi to all
    receivers! This is no. 1"); // adjust
17.   // Check If there is some problem of successfully send
18.   Serial.println(rs.getResponseDescription());
19. }
20.
21. void loop() {
22.     // If something available
23.   if (e220ttl.available()>1) {
24.       // read the String message
25.     ResponseContainer rc = e220ttl.receiveMessage();
26.     // Is something goes wrong print error
27.     if (rc.status.code!=1){
```

Here is the output of module 3 (ADDL = 3). To receive the greetings (“Hi to all receivers! …”) from the other modules, it had to be initialized before them.

```
Hi, I'm going to send message!
Success
Hi to all receivers! This is no. 1
Hi to all receivers! This is no. 2
Hi no. 3, this is an exclusive message from no. 2!
```

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/output_lora_fixed_transmission_no3.png)

Output

Of course, structures can also be sent in fixed transmission mode. However, I try to keep my examples simple and limit them to the relevant topic.

RSSI Received Signal Strength Indicator

As the RSSI function is not available on the E32 modules, this section only applies to the E22 and E220 modules.

The RSSI is a dimensionless number of the byte datatype that tells you how strong the received signal was. The number cannot be directly translated into a signal strength in dBm. To read out the RSSI, simply replace `receiveMessage()` with `ReceiveMessageRSSI()` on the receiver side, for example:

```
ResponseContainer rc = e220ttl.receiveMessageRSSI();
```

Here is an example sketch:

lora_fixed_rssi.ino

```
#include "LoRa_E220.h"

SoftwareSerial mySerial(4,5);
LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Hi, I'm going to send message!");

  // Startup all pins and UART
  e220ttl.begin();

  // Send message
  ResponseStatus rs = e220ttl.sendFixedMessage(0,2,18,"Hello, world?");
```

```
  // Check If there is some problem of successfully send
  Serial.println(rs.getResponseDescription());
}

void loop() {
    // If something available
  if (e220ttl.available()>1) {
      // read the String message
    ResponseContainer rc = e220ttl.receiveMessageRSSI();
    // Is something goes wrong print error
    if (rc.status.code!=1){
```

For the following receiver output, I placed two modules on my desk and sent two messages. For the first message, I used antennas for both modules. Before sending the second message, I disconnected the antenna from the receiving module. As you can see, the *received* signal strength has dropped accordingly. As you can also see, the RSSI is not a measure of the actual signal strength, as this is the same in both cases.

```
Message sent with antenna
RSSI: 223
Message sent without antenna at receiver
RSSI: 184
```

Output lora_fixed_rssi.ino

Using Wake On Radio (WOR)

The power consumption of the LoRa module series E220, E22 and E32 can be significantly reduced if they are operated in WOR mode (“Wake On Radio”). As mentioned above, the transmitter and receiver must have the same WOR period set.

WOR Transmitter

The only additional setting you need to make on the transmitter side is the WOR transmitter mode:

```
e220ttl.setMode(MODE_1_WOR_TRANSMITTER);
```

You do this directly in your application sketch, i.e. not via the configuration sketch. Despite its role as a WOR transmitter, the module can still receive messages.

Here is the sketch for the WOR transmitter:

lora_wor_transmitter.ino

```
#include "LoRa_E220.h"

SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX --
> e220 RX
```

```
LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Hi, I'm going to send a WOR message!");

  e220ttl.begin();
  e220ttl.setMode(MODE_1_WOR_TRANSMITTER);

  // Send message
  ResponseStatus rs = e220ttl.sendFixedMessage(0,2,18,"Hello, world? WOR!");
  // Check If there is some problem of successfully send
  Serial.println(rs.getResponseDescription());
}

void loop() {
    // If something available
  if (e220ttl.available()>1) {
      // read the String message
    ResponseContainer rc = e220ttl.receiveMessage();
    // Is something goes wrong print error
```

The output is still missing. As you can see below, the transmitter receives a confirmation of receipt. This is not a feature of WOR mode, but a feature of the receiver sketch, to which we will come soon.

```
Hi, I'm going to send a WOR message!
Success
We have received the message!
We have received the message!
```

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/output_lora_wor_sender.png)

Output lora_wor_transmitter.ino

Instead of `setMode(MODE_1_WOR_TRANSMITTER)` you use `setMode(MODE_1_WAKE_UP)` for the E32 modules. For the E22 modules, set mode `MODE_1_WOR` in the sketch and define the role as WOR transmitter via the configuration sketch.

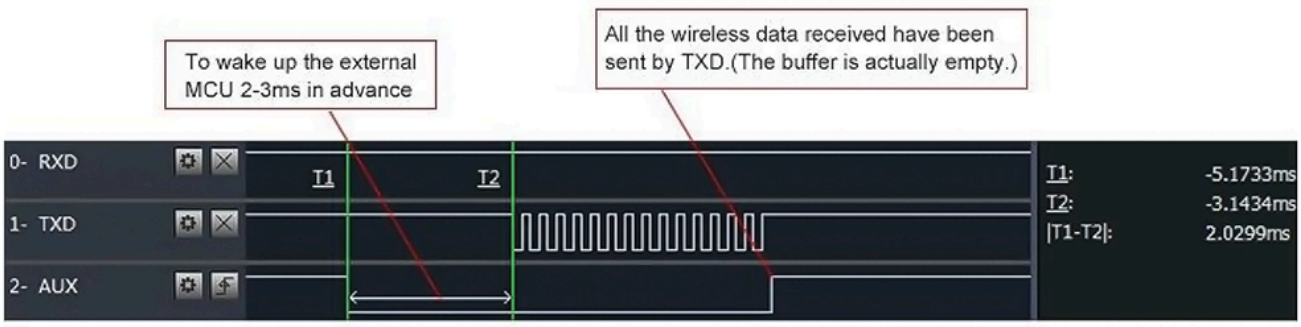
WOR Receiver

You can set the WOR receiver mode as follows:

`e220ttl.setMode(MODE_2_WOR_RECEIVER);`

To be able to send back the confirmation of receipt, you must switch the receiver to normal mode using `e220ttl.setMode(MODE_0_NORMAL);`.

You do not necessarily need the interrupt inserted here, which is triggered by the falling edge of the AUX pin. It is only used to announce the output of the incoming message via the TX pin. Here is the diagram of the behavior of the AUX pin and TX pin:



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/aux_pin_uart_output.png)

AUX pin vs. TX pin when outputting a message via TX

lora_wor_receiver.ino

```
#include "LoRa_E220.h"
#define AUX_PIN 3
volatile bool interruptExecuted = false;

SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX --> e220 RX
LoRa_E220 e220ttl(&mySerial, AUX_PIN, 7, 6); // AUX M0 M1


void wakeUp() {
  interruptExecuted = true;
  //detachInterrupt(digitalPinToInterrupt(AUX_PIN));
}

void setup() {
  Serial.begin(9600);
  delay(500);

  e220ttl.begin();
  e220ttl.setMode(MODE_2_WOR_RECEIVER);

  Serial.println("Start sleep!");
  delay(100);
  attachInterrupt(digitalPinToInterrupt(AUX_PIN), wakeUp, FALLING);
}

void loop() {
  //  If something available
  if (e220ttl.available()>1) {
```

And here is the output:

```
WakeUp Callback, AUX pin go LOW and start receive message!
Message arrived
Hello, world? WOR!
WakeUp Callback, AUX pin go LOW and start receive message!
Message arrived
Hello receiver, greetings from transmitter!
```

Output lora_wor_receiver.ino

Instead of `setMode(MODE_2_WOR_RECEIVER)` you use `setMode(MODE_2_POWER_SAVING)` for the E32 modules. For the E22 modules, set the mode `MODE_1_WOR` in the sketch and define the role as WOR receiver before as a permanent setting via setConfiguration.ino.

Waking up the MCU with WOR

Since the AUX pin goes LOW after / while receiving the message and before outputting it to the microcontroller, this signal can be used to wake up not only the LoRa module but also a sleeping microcontroller via an external interrupt.

Unfortunately, the lead time between the falling edge of the AUX pin and the start of transmission via the TX pin cannot be set (at least I have not found a way to do this). Two to three milliseconds may be too little time, for example, to bring an ESP32 back from deep sleep. On the Arduino Nano, I noticed that at least the first character of the transmitted message was missing. A pragmatic solution, at least for my Arduino Nano configuration, would be to prefix the message with a few dummy characters that can be sacrificed.

I have chosen an alternative solution in my example, namely we first send a “wake-up call”. Before the actual message goes out, we send the transmitter a confirmation that the receiver is awake.

Here is the transmitter sketch:

Transmitter Side

lora_wor_wake_up_mcu_sender.ino

```
1.  #include "LoRa_E220.h"
2.
3.  SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX --
    > e220 RX
4.  LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1
5.
6.  void setup() {
7.    Serial.begin(9600);
8.    delay(500);
9.    Serial.println("Hello, starting now. Type in a message.");
10.
```

```
  e220ttl.begin();
  e220ttl.setMode(MODE_1_WOR_TRANSMITTER);
}

void loop() {
  if (Serial.available()) {
    String input = Serial.readString();
    e220ttl.sendFixedMessage(0,2,18,"......wake up!!"); // wake up call
    e220ttl.setMode(MODE_0_NORMAL); // change to normal mode
    delay(100); // give time for the receiver to wake up

    while(e220ttl.available()<= 1); // wait for confirmation
    ResponseContainer rc = e220ttl.receiveMessage(); // receive message
    rc.status.getResponseDescription();
    Serial.println(rc.data);

    e220ttl.sendFixedMessage(0,2,18,input); // send the actual message
    e220ttl.setMode(MODE_1_WOR_TRANSMITTER); // change back to WOR
```

And here is the unsurprising output:

```
Hello, starting now. Type in a message.
Receiver woke up!
Receiver woke up!
```

Here too – apart from the general adjustments – you must make the setting `setMode(MODE_1_WAKE_UP)` instead of `setMode(MODE_1_WOR_TRANSMITTER)` for the E32 modules. For the E22 modules, set the mode `MODE_1_WOR` in the sketch and define the role as WOR transmitter as a permanent setting.

Receiver Side

The sleep modes of microcontrollers are hardware-specific. I have written an article about the sleep modes of AVR microcontrollers here (https://wolles-elektronikkiste.de/en/sleep-modes-and-power-management). If you are not using an AVR-based microcontroller board, you will have to adapt the sketch accordingly.

lora_wor_wake_up_mcu_receiver.ino

```
#include "LoRa_E220.h"
#include <avr/sleep.h>

#define AUX_PIN 3

SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX --
> e220 RX
LoRa_E220 e220ttl(&mySerial, AUX_PIN, 7, 6); // AUX M0 M1

void wakeUp() {
  delay(0); // add code if you want
}
```

```
void setup() {
  Serial.begin(9600);
  delay(500);

  // Startup all pins and UART
  e220ttl.begin();

  e220ttl.setMode(MODE_2_WOR_RECEIVER);

  Serial.println("Start sleep!");
  delay(100);
  attachInterrupt(digitalPinToInterrupt(AUX_PIN), wakeUp, FALLING);
  set_sleep_mode(SLEEP_MODE_PWR_DOWN); // define sleep mode
}
```

Here is the output:

```
Start sleep!
......wake up!!
Hello, this is a message from the transmitter!
......wake up!!
Just another message.
```

(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/outputlora_wor_wake_up_mcu_receiver-1.png)

Output lora_wor_wake_up_mcu_receiver.ino

If you change the WORPeriod, you must change the delays in the sketches so that the interaction works.

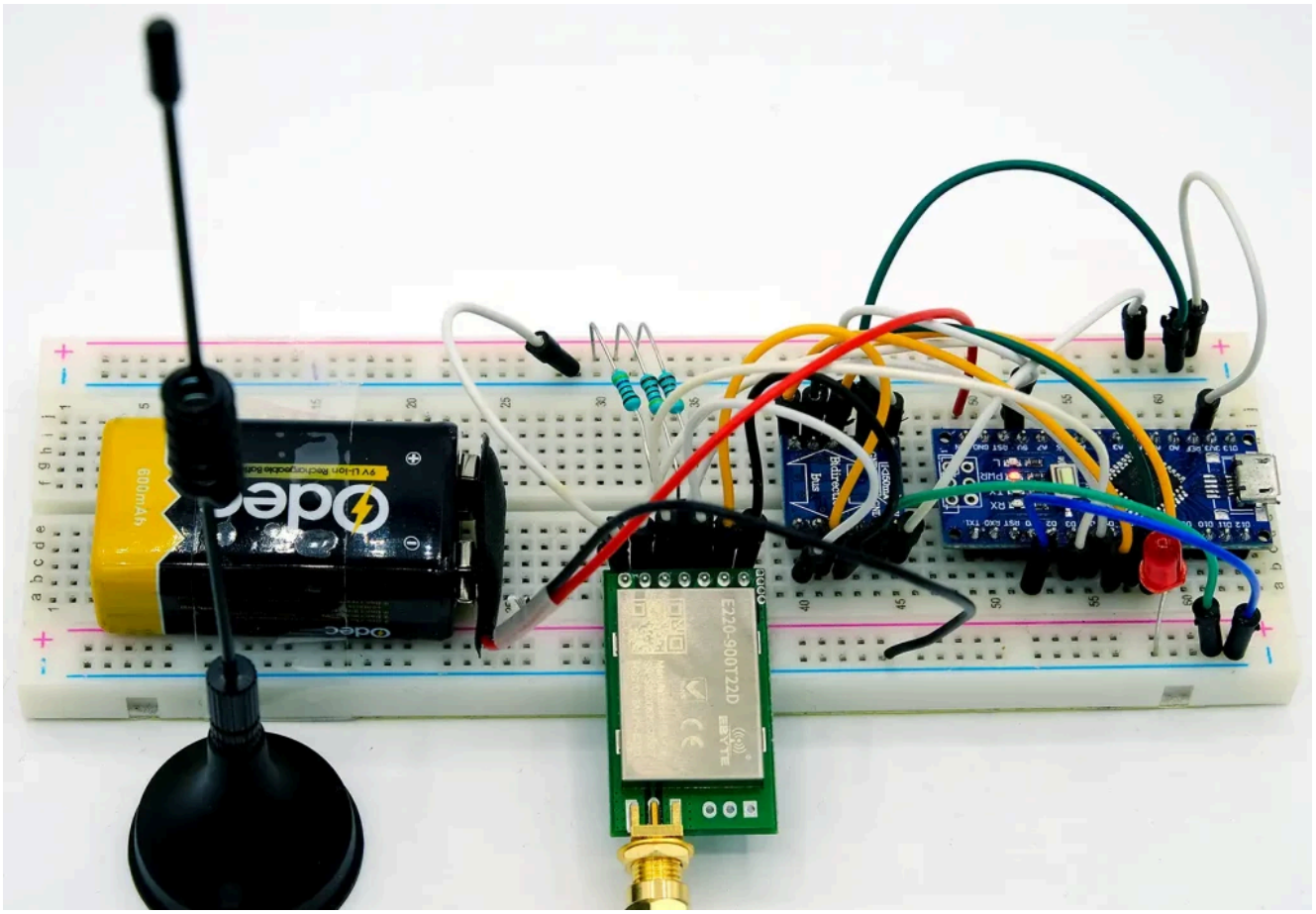
For E32 modules, among other things, `setMode(MODE_2_WOR_RECEIVER)` must be replaced by `setMode(MODE_2_POWER_SAVING)`. For the E22 modules, set the mode `MODE_1_WOR` in the sketch and define the role as WOR receiver as a permanent setting.

Range Test

I carried out a range test with the E220-900T22D module. According to the data sheet, the range is up to 5 km, but only if there is a clear line of sight between the modules, which is rather rare at this distance.

I selected the highest transmission power of 22 dBm (one-off test that exceeds the allowed limits in Germany) and the lowest data rate because this should provide the greatest range. The antenna used was an 868 MHz antenna. The Arduino was supplied with power via a 9 V lithium battery. Measures for voltage stabilization such as additional capacitors were not used.

Here is the receiver unit:



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/distance_test_setup-1024x708.webp)

LoRa E220 range test setup (receiver)

The transmitter has sent a message every 5 seconds:

lora_range_test_transmitter.ino

```
#include "LoRa_E220.h"

SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX --> e220 RX
LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Hi, I'm going to send message!");

  // Startup all pins and UART
  e220ttl.begin();

  // Send message
  ResponseStatus rs = e220ttl.sendFixedMessage(0,2,18,"Hi to receiver!"); // adjust
  // Check If there is some problem of successfully send
  Serial.println(rs.getResponseDescription());
```

```
    delay(2000);
  }

  void loop() {
    // If something available
    static unsigned long lastSend = 0;
    if (millis() - lastSend > 5000) {
      e220ttl.sendFixedMessage(0,2,18, "Hi Receiver, did you get this
message?");
```

The receiver has received the message, checked the content and, if correct, briefly illuminated an LED on pin 8 five times.

lora_range_test_receiver.ino

```
#include "LoRa_E220.h"
#define LED_PIN 8

SoftwareSerial mySerial(4, 5); // Arduino RX <-- e220 TX, Arduino TX --
> e220 RX
LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
  delay(500);

  Serial.println("Hi, waiting for messages!");

  // Startup all pins and UART
  e220ttl.begin();
  delay(2000);
}

void loop() {
  // If something available
  if (e220ttl.available()>1) {
    ResponseContainer rc = e220ttl.receiveMessage();
    //Serial.println(rc.data);
    if(rc.data = "Hi Receiver, did you get this message?"){
      for(int i=0; i<5; i++){
        digitalWrite(LED_PIN, HIGH);
        delay(100);
        digitalWrite(LED_PIN, LOW);
```

The transmitter was placed on my desk at home. I went out into the fields with the receiver unit and checked up to what distance I could receive messages. The radio signal first had to penetrate my own house wall, then a narrow strip of woodland and three or four houses before it crossed the open field. The terrain was flat. This gave me a range of 1.34 km.



(https://wolles-elektronikkiste.de/wp-content/uploads/2024/03/range_test-1024x142.webp)

Result of the range test (measured with Google Maps).

Appendices – Transceiver Sketches with Settings for E220, E22, E32

Appendix 1 – E220 Transceiver Sketch with Settings

Sketch with embedded configuration for E220 modules. You can copy the function `setConfiguration()` and use it in other sketches. The sketch was tested with two E220-900T22D modules.

lora_e220_fixed_transmission_with setting_options.ino

```
// #define FREQUENCY_433 // default value without set
// #define FREQUENCY_170
// #define FREQUENCY_470
#define FREQUENCY_868
// #define FREQUENCY_915
// #define E220_22 // default value without set
// #define E220_30 // uncomment in case you use an E220...T30D or E220...T30S
#include "LoRa_E220.h"

SoftwareSerial mySerial(4,5);
LoRa_E220 e220ttl(&mySerial, 3, 7, 6); // AUX M0 M1

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Beginning configuration...");
  e220ttl.begin();
  setConfiguration();
  Serial.println("Hi, I'm going to send message!");

  ResponseStatus rs = e220ttl.sendBroadcastFixedMessage(18, "Hello, world?");
  Serial.println(rs.getResponseDescription());
}

void loop() {
    // If something available
```

Appendix 2 – E22 Transceiver Sketch with Settings

Sketch with embedded configuration for E22 modules. You can copy the function `setConfiguration()` and use it in other sketches. The sketch was tested with two E22-400T22D modules.

lora_e22_fixed_transmission_with setting_options.ino

```
// #define FREQUENCY_433 // default value without set
// #define FREQUENCY_170
// #define FREQUENCY_470
// #define FREQUENCY_868
// #define FREQUENCY_915
// #define E22_22 // default value without set
// #define E22_30 // uncomment in case you use an E22...T30D or E22...T30S
#include "LoRa_E22.h"

SoftwareSerial mySerial(4,5);
LoRa_E22 e22ttl(&mySerial, 3, 7, 6); // AUX M0 M1

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Beginning configuration...");
  e22ttl.begin();
  setConfiguration();
  Serial.println("Hi, I'm going to send message!");

  ResponseStatus rs = e22ttl.sendBroadcastFixedMessage(23, "Hello, world?");
  Serial.println(rs.getResponseDescription());
}

void loop() {
    // If something available
```

Appendix 3 – E32 Transceiver Sketch with Settings

Sketch with embedded configuration for E32 modules. You can copy the function `setConfiguration()` and use it in other sketches. The sketch was tested with two E220-900T22D modules.

lora_e32_fixed_transmission_with setting_options.ino

```
// #define FREQUENCY_433 // default value without set
// #define FREQUENCY_170
// #define FREQUENCY_470
// #define FREQUENCY_868
// #define FREQUENCY_915/* Choose your module */
// #define E32_TTL_100
```

```
// #define E32_TTL_500
#define E32_TTL_1W // E32-TTL-1W, E32-433T30S/D, E32-868T30S/D, E32-915T30S/D
// #define E32_TTL_2W
#include "LoRa_E32.h"

SoftwareSerial mySerial(4,5);
LoRa_E32 e32ttl(&mySerial, 3, 7, 6); // AUX M0 M1

void setup() {
  Serial.begin(9600);
  delay(500);

  Serial.println("Beginning configuration...");
  e32ttl.begin();
  setConfiguration();
  Serial.println("Hi, I'm going to send message!");

  ResponseStatus rs = e32ttl.sendBroadcastFixedMessage(23, "Hello, world?");
  Serial.println(rs.getResponseDescription());
}
```

Acknowledgement

I would like to thank Renzo Mischianti (https://mischianti.org/) for the wonderful libraries and the detailed tutorials.

Posted in 433 MHz (https://wolles-elektronikkiste.de/en/category/wireless/433-mhz-en), Wireless (https://wolles-elektronikkiste.de/en/category/wireless) Tagged Arduino (https://wolles-elektronikkiste.de/en/tag/arduino-en-2), circuit (https://wolles-elektronikkiste.de/en/tag/circuit-en-2), E22 (https://wolles-elektronikkiste.de/en/tag/e22-en), E220 (https://wolles-elektronikkiste.de/en/tag/e220-en), E32 (https://wolles-elektronikkiste.de/en/tag/e32-en), Ebyte (https://wolles-elektronikkiste.de/en/tag/ebyte-en), FEC (https://wolles-elektronikkiste.de/en/tag/fec-en), Fixed Transmission (https://wolles-elektronikkiste.de/en/tag/fixed-transmission-en), LBT (https://wolles-elektronikkiste.de/en/tag/lbt-en), Long Range (https://wolles-elektronikkiste.de/en/tag/long-range-en), LoRa (https://wolles-elektronikkiste.de/en/tag/lora-en), LPWAN (https://wolles-elektronikkiste.de/en/tag/lpwan-en), Range (https://wolles-elektronikkiste.de/en/tag/range-en), RSSI (https://wolles-elektronikkiste.de/en/tag/rssi-en), settings (https://wolles-elektronikkiste.de/en/tag/settings-2), Structures (https://wolles-elektronikkiste.de/en/tag/structures), Transparent transmission (https://wolles-elektronikkiste.de/en/tag/transparent-transmission-en), Wake on Radio (https://wolles-elektronikkiste.de/en/tag/wake-on-radio-en), WOR (https://wolles-elektronikkiste.de/en/tag/wor-en)

← Character Arrays vs. Strings (https://wolles-elektronikkiste.de/en/character-arrays-vs-strings)

Serial and SoftwareSerial → (https://wolles-elektronikkiste.de/en/serial-and-softwareserial)

45 thoughts on “Using LoRa with the EByte E220, E22 and

E32 series”

Andrei **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-42775)** *August 15, 2026*

Hi,
When I try to compile a sketch “lora_transparent.ino” for ESP32 I get error
“‘SoftwareSerial’ does not name a type; did you mean ‘HardwareSerial’? ”
Could you please explain which SoftwareSerial library I should use?

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-42781)** *August 16, 2026*

Hi Andrei,

there is a SoftwareSerial library available, which I haven’t tried (https://github.com/plerup/espsoftwareserial (https://github.com/plerup/espsoftwareserial)).

But I would recommend using another HardwareSerial instead, e.g. Serial2. You find the constructors in the example sketches of the library, e.g. here:

https://github.com/xreef/EByte_LoRa_E220_Series_Library/blob/master/examples/02_sendTransparentTransmission/02_sendTransparentTransmission.ino#L62 (https://github.com/xreef/EByte_LoRa_E220_Series_Library/blob/master/examples/02_sendTransparentTransmission/02_sendTransparentTransmission.ino#L62)

Regards, Wolfgang

Reply

Andrei **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-42788)** *August 17, 2026*

Thanks
I’ll try
BR,
Andrei

Reply

Pascal **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-41815)** *April 30, 2026*

Hi,

I'm trying to set up communicatioon (FIXED_TRANSMISSION) for a RC boat with your library. Everything seems OK, except for one point, after configuring the receiver (setConfiguration), and trying to read a structured message, it seems, there are some bytes as a message (not send by me) : 0xFFFFFFC0 ADDRL ADDRH 0x1A CHAN 0xFFFFFFC4

Seems to be an echo of my configuration, but myReceiveMessage(sizeof(mystruct)) failed with "wrong size";
Do I need to "getConfiguration" after "setConfiguration" to get rid of this message ?

Thanks

Reply

Pascal **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-41816)** *April 30, 2026*
OK, after decoding those bytes, it's the configuration sequence (with defaults values, except for ADDRH, ADDRL, CHAN et FIXED_TRANSMISSION).

So, why after loramodule.setconfiguration(…); those bytes are in the reception buffer ? Do I miss something ?

Thanks

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-41819)** *April 30, 2026*
Hi, first of all, it's not my library, but Renzo Mischianti's library. And it's not one library but different ones depending on the module you use. The libraries for the E220, E22 and E32 series have examples for sending structures with the fixed transmission mode:

https://github.com/xreef/EByte_LoRa_E220_Series_Library/blob/master/examples/05_sendFixedTransmissionStructure/05_sendFixedTransmissionStructure.ino (https://github.com/xreef/EByte_LoRa_E220_Series_Library/blob/master/examples/05_sendFixedTransmissionStructure/05_sendFixedTransmissionStructure.ino)

https://github.com/xreef/EByte_LoRa_E22_Series_Library/tree/master/examples/esp32_e22_05_SendFixedTransmissionStructure (https://github.com/xreef/EByte_LoRa_E22_Series_Library/tree/master/examples/esp32_e22_05_SendFixedTransmissionStructure)

https://github.com/xreef/LoRa_E32_Series_Library/tree/master/examples/sendReceiveFixedMessageStructure (https://github.com/xreef/LoRa_E32_Series_Library/tree/master/examples/sendReceiveFixedMessageStructure)

So I would start with the sketch that belongs to the series that you are using. If it works, then I would take it as a basis for your project. If the example does not work, and you are sure you did everything correctly then you might consider raising this as an issue on GitHub.

I also recommend reading Renzo's tutorial for the module series you are using. I tried to cover everything in one article, whereas Renzo has written eight or nine articles per series.

Regards, Wolfgang

Reply

> ***Pascal*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-41821)** *April 30, 2026*
> Thanks
>
> Examples works fine, the only issue is when I change the configuration and run the receive Loop without rebooting the card.
>
> I will check the Renzo's tutorials.
> Thanks
>
> Reply
>
> > ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-41823)** *May 1, 2026*
> > OK, understand. Basically, I like solving problems, but, unfortunately, I won't have time in the coming days to dive deeper into this. If you can't solve it, please come back and I will look at it later.
> >
> > Reply

MIRK **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-40645)** *February 25, 2026*
can e32 nad e22 communicate to each other? if yes then pleae advise how. I am not able to do that.

Reply

> ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-40653)** *February 25, 2026*
> Renzo Mischianti, who has dived into these modules much deeper than me, says they are not compatible:
>
> https://mischianti.org/forums/topic/lora-e32-e22-e220-module-compatibility/ (https://mischianti.org/forums/topic/lora-e32-e22-e220-module-compatibility/)
>
> I have not tried it myself, but it seems your experience is in line with Renzo's statement.

Reply

iotebyte (https://www.cdebyte.com) **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-40271)** *February 7, 2026*

Huge thanks for putting out this awesome technical article! 👏 We're the official team behind EBYTE.

If you want more tutorials and guides for our various product models, head over to our official website: cdebyte.com

Plus, we regularly introduce new product series on a monthly basis. To explore our latest offerings, please visit:
👉 https://www.cdebyte.com/Aboutus-NewProductExpress (https://www.cdebyte.com/Aboutus-NewProductExpress)

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-40277)** *February 7, 2026*

Thank you for your feedback and for providing the link. I had already had a brief look. I will definitely visit your site more often!

Reply

iot ebyte (https://ebyteiot.com) **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-42867)** *August 24, 2026*

Thank you very much for your continued interest in and support of our products. For more product details and technical specifications, please visit our official store at ebyteiot.com. Free samples are available for developers!

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-42873)** *August 24, 2026*

Thank you!

Reply

Andrzej Wysocki **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-39389)** *December 21, 2025*

Bardzo dobrze napisany artykuł. Bardzo mi pomógł w projekcie który robię z użyciem modułów LORA E220.
Dziękuję!!! 🙂

Translation added by Wolfgang: *Translation: A very well-written article. It has been very helpful in the project I am working on using LORA E220 modules.*
Thank you!

Reply

> ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-39410)** *December 22, 2025*
> Thank you! Dziękuję!
>
> Reply

Andrey **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-39016)** *November 29, 2025*
Thanks for very good description.
Tell me, please, have you ever worked with the eByte configurator? It is listed in the E220-400T22D User Manual (433/470MHz 22dBm LoRa Wireless Module), but it is not entirely clear how to connect and configure the module.

Reply

> ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-39026)** *November 30, 2025*
> Hi Andrey, I have not tried it. But I have asked ChatGPT how to connect. Here is the answer:
>
> 1) Required hardware
> • USB↔TTL adapter with 3.3 V operating voltage (FTDI / CP2102 / CH340). Not 5 V!
> • If the adapter cannot supply a stable 3.3 V: use an external 3.3 V regulator (e.g. AMS1117-3.3).
> 2) Wiring (typical connections)
> Adapter → EByte:
> • 3.3V (adapter VCC 3.3 V) → VCC module
> • GND → GND module (common ground!)
> • TX (adapter) → RXD (module)
> • RX (adapter) → TXD (module)
> Important: Cross TX ↔ RX. Many connection problems arise from this or from a missing common ground.
> 3) M0 / M1 for configuration
> For the E22/E220 series, the mode pins must be set so that the module switches to configuration mode. For E22/E220, this is usually:

• M0 = LOW (GND) and M1 = HIGH (3.3 V) → Configuration mode (UART controller at 9600, 8N1 for configuration commands).
(Other combinations = Normal / WOR / Deep Sleep — see manual).

Tip: Many breakout boards have solder bridges/jumpers for M0/M1 (short/open). If you don't want to solder the pins permanently, you can use jumpers/plugs.
4) Software / serial settings
• Download the official configuration programme (RF_Setting / RF_Settingv3.x) from eByte / your retailer's website.
• Before opening the tool: put the module into configuration mode (M0=0, M1=1), then select the COM port.
• In configuration mode, the module usually expects 9600 baud, 8N1 (some variants use this; the manual specifies 9600 in Config Mode). If the tool reports "module not response", M0/M1, baud or COM port are often incorrect.

Since I have not tried it I can't promise that it works. Sometimes ChatGPT is providing wrong information.

Reply

iotebyte (https://www.cdebyte.com) **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-40272)** *February 7, 2026*
You can contact service@cdebyte.com (mailto:service@cdebyte.com) for technical support.

Reply

Rees **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-32791)** *April 21, 2025*
fair play thats a comprehensive tutorial 👏👏

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-32801)** *April 22, 2025*
Thank you!

Reply

DJ **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-32290)** *March 13, 2025*
Hi,

I am looking for a gateway device, which is able to communicate with E22-400T22S, not expensive, easy to program, has interface (preferably USB-Serial) to a normal computer.
Do you have any recommendation?

Reply

> ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-32299)** *March 13, 2025*
> I don't have experience with these devices. Therefore, I can't recommend any. Maybe someone else who reads this?
>
> Reply

emre **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-31771)** *February 11, 2025*
I have 2 Teensy 4.1 and 2 LoRa E22 900T22D modules. Can I make them work in both transmitter and receiver mode?

For example, one LoRa module will continuously send sensor data, while the other LoRa module will receive data from the serial monitor and send it to the sensor-transmitting LoRa module. The sensor-transmitting LoRa module will then receive this command.

Will this setup work?

Reply

> ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-31772)** *February 11, 2025*
> Hi, why should it not work? I don't have experience with Teensy Boards, but I don't expect problems that can't be solved. The E22 transceiver sketch in the appendix should be a good starting point. You may need to do few adjustments, e.g. with regards to the pins. If you want to deep dive into the E22 in particular, I recommend you also take a look into Renzo Mischianti's articles:
> https://mischianti.org/ebyte-lora-e22-device-for-arduino-esp32-or-esp8266-specs-and-basic-usage-1/ (https://mischianti.org/ebyte-lora-e22-device-for-arduino-esp32-or-esp8266-specs-and-basic-usage-1/)
> Just try! Good luck!
>
> Reply

Paul D. Wilkie (http://self) **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-31700)** *February 5, 2025*

I have been working with EBYTE's E32 915T30D LoRa radios; however they now offer a replacement, the E323 900T30D. However, this replacement radio has limited the AirRate to 2.4K which limits the range in non line of site situations (none line of sight situations, dense forest trails, with steep terrain).

One of my forest test areas provided ranges of 8K – 10K feet or more with the E32 915T30D LoRa radios with AirRate set @ 300.

With the replacement E32 900T30D radios, which limits the AirRate @ 2.4k, I record less that 4000 feet in the same area of testing?

Do you happen to know a source for these now obsolete EBYTE's E32 915T30D LoRa radios?

Reply

> ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-31704)** *February 5, 2025*
> It seems the E32 915T30D module is hard get. It seems it is avialable on Alibaba: https://www.alibaba.com/product-detail/Module-Long-distance-E32-915T30D-SX1278_1600130443546.html (https://www.alibaba.com/product-detail/Module-Long-distance-E32-915T30D-SX1278_1600130443546.html)
> But I have no experience with Alibaba. I think it's primarily a business to business shop.
> All EByte modules with higher sending power use a different wavelength and are only available as SMD.
>
> Reply

YUSUF EREN **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-29772)** *September 23, 2024*
Thank you for excellent documantation,
I have a simple LoRa project, and I am using two LoRa E22 400 TD modules to communicate, along with two ESP32s. The reason for this choice is to enable sending the received data to the web via Wi-Fi. However, I haven't found any examples related to E22 and ESP32, and I am unable to initialize the LoRa modules. What do you think could be the reason for this?

Reply

> ***Wolfgang Ewald*** **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-29775)** *September 23, 2024*
> Hi, it could be related to the fact that SoftwareSerial does not on the ESP32. To be more exact, SoftwareSerial is not part of the Arduino ESP32 board package. You could use Serial2 instead, like this:
> LoRa_E22 e22ttl(&Serial2);
> On Renzo Mischianti's great website, you find a complete sketch for the E22 module connected to an ESP32. Here's the link:
> https://mischianti.org/ebyte-lora-e22-device-for-arduino-esp32-or-esp8266-specs-and-

basic-usage-1/ (https://mischianti.org/ebyte-lora-e22-device-for-arduino-esp32-or-esp8266-specs-and-basic-usage-1/)
Good luck!

Reply

tenshi **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-29515)** *September 2, 2024*
help me do everything as in your example, I betray 3 int numbers and the receiver freezes at 132 reception, I transmit 1 int variable and the receiver freezes at 352 reception how to fix it is that you are not looking for or is there a problem????

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-29529)** *September 3, 2024*
Hi, which example have you used, lora_struct_transmitter.ino / lora_struct_receiver.ino ? I understand your message consists of 3 integers, so what have you modified? Which module are you using and what is your microcontroller board? If you want, you can send me your sketches (wolfgang.ewald@wolles-elektronikkiste.de) and I hvae a look. And what happens if you take my sketches without any changes? Do they also freeze after a certain time?

Reply

William Lucid (https://weather-observations.live/index.php) **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-28955)** *July 20, 2024*
Wanted say thank for the settings you provided for the tinySA Ultra; I was able to confirm the WOR waveform ws present to continue with theE220-Remote-Switch project and complete the project. Appreciate your writeup and experieces with the E220!

Project post on https://esp32.com (https://esp32.com): https://esp32.com/viewtopic.php?f=17&t=40930 (https://esp32.com/viewtopic.php?f=17&t=40930)

Completed project code: https://github.com/Tech500/E220-Remote-Switch (https://github.com/Tech500/E220-Remote-Switch)

Best Regards,
William

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-28989)** *July 24, 2024*
Thank you for the feedback!

Reply

William (https://weather-observations.live/index.php) **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-28573)** *June 24, 2024*

Thank you for the very, excellent documentation.

Please share tinySA Ultra cofiguration for Ebyte E220-900T30D. did you use triggering? Have watched Eric's youtube videos, feeling a little overwelmed; especially with triggering.

[url=https://drive.google.com/file/d/16ueK_zGvQOtzUgejC9PCGnO1Dyy_-rdm/view?usp=sharing]I managed once to capture preamble and message.[/url]

My settings were not recorded; best I can recall: Start 916.1 Mhz., Stop 916.3 Mhz,. Center 916.125 Mhz,. Span .300 Mhz. Used tinySA desktop app and OBS to capture video.

Wonder if "the smoke is out"of module. Able to config modules using Ebyte RF software without any issue.

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-28583)** *June 25, 2024*

Hi William,
I agree, there's lots of options for the setting. I spent quite some time until I got what I wanted. Fortunately, there is an option to save the settings. Since I do not use this nice device every day, I will have forgotten next time how I got it working.
So the main settings are:
Frequency: Start: 845 MHz / Stop: 900 MHz
Level: Auto
Trigger: Normal / Level: -60 dBm / Edge: Down / Trigger: PRE / Interval: 0s
Trace: Enable
Measure: – 3dB WIDTH
Mode: HIGH in

What I noticed is that one should not set the frequency limits too wide. I don't say it's the optimum settings – it's just what works for me.

Hope this helps!

Reply

Daniel **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-28094)** *May 22, 2024*

Is it possible that module E32 , E22 and E220 will work together. What I need to setup to be able send and revive packages on all of them

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-28098)** *May 22, 2024*

Hi Daniel,

probably not, see also here:
https://mischianti.org/forums/topic/lora-e32-e22-e220-module-compatibility/ (https://mischianti.org/forums/topic/lora-e32-e22-e220-module-compatibility/)
But not tested by myself.
Regards, Wolfgang

Reply

Zen **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27455)** *March 31, 2024*

Are you able to send the full payload size when in fixed point transmission mode? I've been testing and the modem seems to consider the first 3 bytes as part of the payload when they aren't received on the other end.

I've also noticed that in order to send multiple packets, the payload must have the address/channel at every new subpacket, the transmission while in transparent mode seems a lot easier to operate considering this.

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27465)** *April 2, 2024*

I have tested this now with an E22 module, which has a maximum payload size of 240 bytes. Indeed, I was only able to send 237 bytes, the rest is used for address and channel as you rightly say. Thanks for the useful comment!

Reply

Zen **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27473)** *April 2, 2024*

Thanks for checking my work, these modules have some interesting undocumented behavior.

I wonder if that data is really transmitted as part of the payload, and if the receiving device uses that "header" info to know if it should handle the rest of the frame. That still doesn't explain why the channel would be part of that, and I'm honestly a bit confused why this has to be specified when I don't think it sets the receiving channel. I can see some use in sending a packet across each channel or something, but then it would still only be ready to receive on a single channel.

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27474)** *April 2, 2024*
I also don't know why the channel should be sent. However, the three bytes are used for the channel and the address. It's mentioned in the data sheet.

Reply

Zen **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27425)** *March 28, 2024*
Is there any way to know the address a packet was received from while in fixed point mode?

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27431)** *March 28, 2024*
The sender address is – to my best knowledge – not sent. A pragmatic solution would be if you add the address to the message you.

Reply

Zen **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27440)** *March 29, 2024*
Thanks, I bought a few of these modules a few weeks ago, and I believe some rylr 998 modules were aware of the receive address, but are a bit less compatible.

I've tested the e22 and e220's talking to each other, the only issue is that the e22's have the option for a network ID while the 220's don't seem to have that.

The maximum subpacket size is 240b on the e22 and 200b on the e220, so encoding the sender address in here seems like a waste of space and potentially unreliable. I'm wondering if the SPI versions reveal info about the sender.

Reply

Wolfgang Ewald **(https://wolles-elektronikkiste.de/en/using-lora-with-the-ebyte-e220-e22-and-e32-series#comment-27443)** *March 30, 2024*
I have not tested the SPI type versions (yet).

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐ **Ja, informiere mich über neue Beiträge! Inform me about new posts! You will NOT be notified about new or follow-up comments!**

Don't subscribe **E-Mail-Benachrichtigung bei weiteren Kommentaren / E-mail notification about new comments**
Options: Nicht abonnieren: Don't subscribe, Alle = All, Antworten auf meine Kommentare = Only replies to my comments
Oder / Or: Abo ohne Kommentar / Subscription without comment (https://wolles-elektronikkiste.de/en/comment-subscriptions/?srp=20804&srk=f4c5b49202dccf343ed27773685414c7&sra=s&srsrc=f).

Post Comment

ABO / SUBSCRIPTION?

Vorname

E-Mail *

Zur Datenschutzerklärung. (http://wolles-elektronikkiste.de/privacy-policy)

Abonniere / subscribe!

RECENT POSTS

- Using ArduinoBLE (https://wolles-elektronikkiste.de/en/using-arduinoble)
- BLE with the ESP32 – An Introduction (https://wolles-elektronikkiste.de/en/ble-with-the-esp32-an-introduction)
- TinyCore board package for classic ATtinys (https://wolles-elektronikkiste.de/en/tinycore-board-package-for-classic-attinys)
- Programming AVR MCUs with MiniCore (https://wolles-elektronikkiste.de/en/programming-avr-mcus-with-minicore)
- BNO055 9-DoF sensor (https://wolles-elektronikkiste.de/en/bno055-9-dof-sensor)

RECENT COMMENTS

- Wolfgang Ewald on Interrupts – Part 2: Pin Change Interrupts (https://wolles-elektronikkiste.de/en/interrupts-part-2-pin-change-interrupts#comment-43380)
- Roberto on Interrupts – Part 2: Pin Change Interrupts (https://wolles-elektronikkiste.de/en/interrupts-part-2-pin-change-interrupts#comment-43373)
- kms on tinyAVR Series Part 2: Timer A/B, CCL, SLPCTRL (https://wolles-elektronikkiste.de/en/tinyavr-series-part-2-timer-a-b-ccl-slpctrl#comment-43351)
- Wolfgang Ewald on tinyAVR Series Part 2: Timer A/B, CCL, SLPCTRL (https://wolles-elektronikkiste.de/en/tinyavr-series-part-2-timer-a-b-ccl-slpctrl#comment-43350)
- kms on tinyAVR Series Part 2: Timer A/B, CCL, SLPCTRL (https://wolles-elektronikkiste.de/en/tinyavr-series-part-2-timer-a-b-ccl-slpctrl#comment-43348)

ARCHIVES Select Month

THEMEN

- Boards and Microcontrollers (https://wolles-elektronikkiste.de/en/category/boards-and-microcontrollers)
- English (https://wolles-elektronikkiste.de/en/category/lang-en-en)
- Other parts (https://wolles-elektronikkiste.de/en/category/other-parts)
  - Port expansion (https://wolles-elektronikkiste.de/en/category/other-parts/port-expansion)
- Other stuff (https://wolles-elektronikkiste.de/en/category/other-stuff)
- Projects (https://wolles-elektronikkiste.de/en/category/projects)
- Sensors (https://wolles-elektronikkiste.de/en/category/sensors)
  - Acceleration (https://wolles-elektronikkiste.de/en/category/sensors/acceleration)
  - Current, voltage (https://wolles-elektronikkiste.de/en/category/sensors/current-voltage)
  - Distance, light, movement (https://wolles-elektronikkiste.de/en/category/sensors/distance-light-movement)
- Software and tools (https://wolles-elektronikkiste.de/en/category/software-and-tools)
- Sound (https://wolles-elektronikkiste.de/en/category/sound-en)
- Uncategorized (https://wolles-elektronikkiste.de/en/category/uncategorized)
- wireless (https://wolles-elektronikkiste.de/en/category/wireless-en-2)
- Wireless (https://wolles-elektronikkiste.de/en/category/wireless)
  - 433 MHz (https://wolles-elektronikkiste.de/en/category/wireless/433-mhz-en)
  - Bluetooth (https://wolles-elektronikkiste.de/en/category/wireless/bluetooth-en)
  - Infrared (https://wolles-elektronikkiste.de/en/category/wireless/infrared)
  - WLAN (https://wolles-elektronikkiste.de/en/category/wireless/wlan-en)

FORMALES / FORMAL STUFF

- Legal notice (https://wolles-elektronikkiste.de/en/legal-notice)

ANDERES / OTHER STUFF

- Home (https://wolles-elektronikkiste.de/?lang=en)
- Visit me on Github (https://github.com/wollewald)

Designed by Inkhive Web Design (https://inkhive.com/). © 2026 Wolles Elektronikkiste. All Rights Reserved.

Multilingual WordPress (https://wpml.org/?utm_source=wpml-plugin&utm_medium=credit&utm_campaign=credit-footer&wpv=5.1.0) with WPML